Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase V · Alineamiento, aprendizaje por refuerzo, agentes e interpretabilidad

Módulo 20

Ajuste fino e instrucciones

Ajuste supervisado, olvido catastrófico y adaptación eficiente en parámetros

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 20: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
θ_0, θ	parámetros del modelo preentrenado y parámetros ajustados
$u = u_{1:n}$, $y = y_{1:m}$	instrucción (*prompt*) y respuesta, como secuencias de tokens
$x_{1:T}$, $m_t \in \{0,1\}$	secuencia completa (instrucción seguida de respuesta) y máscara de las posiciones que puntúan
$\mathcal{L}_{\mathrm{SFT}}$	pérdida del ajuste supervisado (entropía cruzada enmascarada)
η, η^l	tasa de aprendizaje global y de la capa l
λ	intensidad de la penalización que ancla los parámetros a θ_0
$\mathcal{I}(\theta)$, F_i	matriz de información de Fisher y su diagonal
D, d'	número total de parámetros y dimensión del subespacio entrenable (tema 4)
$W \in \mathbb{R}^{d\times k}$, $x \in \mathbb{R}^k$	en los temas 4 a 6, matriz preentrenada congelada y entrada de la capa
$A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$	factores de LoRA; r es el rango
α, $s = \alpha/r$	hiperparámetro de escala de LoRA y factor efectivo
$\sigma_1 \geq \sigma_2 \geq \cdots$	valores singulares
Φ, Φ^{-1}	función de distribución de la normal estándar y su inversa (función cuantil)
N, N_{LoRA}	parámetros del modelo base y de los adaptadores

Índice

Sobre este resumen **ii**

1 Transferencia y ajuste fino **1**
1.1 Modelo base, tarea de origen y tarea de destino . . . 1
1.2 Extracción de características y ajuste completo . . . 1
1.3 ULMFiT: ajuste fino de un modelo de lenguaje . . . 2
1.4 Del ajuste por tareas al posentrenamiento . . . 3

2 Ajuste supervisado con instrucciones **4**
2.1 Datos de instrucciones . . . 4
2.2 La pérdida enmascarada . . . 5
2.3 Lotes: relleno, normalización, empaquetado y acumulación . . . 6
2.4 La plantilla de chat . . . 6
2.5 ¿Cuántos datos hacen falta? La alineación superficial . . . 7

3 Olvido catastrófico **8**
3.1 El fenómeno . . . 8
3.2 Un modelo cuadrático del compromiso . . . 8
3.3 Consolidación elástica de pesos . . . 9
3.3.1 Justificación bayesiana . . . 9
3.3.2 Información de Fisher . . . 9
3.4 Otros remedios . . . 10

4 Ajuste eficiente en parámetros **11**
4.1 Planteamiento . . . 11
4.2 Adaptadores . . . 11
4.3 Ajuste de *prompt* y de prefijo . . . 12
4.4 Dimensión intrínseca . . . 13

5 LoRA: adaptación de bajo rango **14**
5.1 Definición y número de parámetros . . . 14
5.2 Rango de la corrección . . . 15
5.3 Gradientes e inicialización . . . 15
5.4 La escala α/r . . . 15
5.5 Por qué funciona: relación con la SVD . . . 16

6 QLoRA y elección de estrategia **17**

6.1 Cuantización por bloques 17

6.2 El formato NF4 17

6.3 Doble cuantización 18

6.4 QLoRA 18

6.5 ¿Ajustar, dar instrucciones en el contexto o recuperar documentos? 19

Bibliografía **20**

Tema 1

Transferencia y ajuste fino

Este tema formula el ajuste fino como un problema de optimización que parte de un modelo ya entrenado y presenta las técnicas clásicas de transferencia que preceden al posentrenamiento de los modelos de lenguaje actuales. La distinción entre extracción de características y ajuste completo sigue el §14.2 de Zhang *et al.* [28]; las técnicas de ULMFiT se toman del artículo de Howard y Ruder [17, §3.2 y §3.3], y la visión del posentrenamiento, del capítulo 8 de Jurafsky y Martin [19] y de la clase 8 de CS224N [5].

1.1 Modelo base, tarea de origen y tarea de destino

Definición 1.1 (Aprendizaje por transferencia). Sea f_θ una familia de modelos con parámetros $\theta \in \mathbb{R}^D$. Se llama *tarea de origen* a la minimización de una pérdida $\mathcal{L}_{\text{orig}}$ sobre un conjunto de datos grande, y *modelo preentrenado* a unos parámetros θ_0 que la minimizan aproximadamente. Dada una *tarea de destino* con datos $\mathcal{D}$ y pérdida $\mathcal{L}_{\mathcal{D}}$, el *ajuste fino* consiste en minimizar aproximadamente $\mathcal{L}_{\mathcal{D}}$ mediante un método iterativo de primer orden inicializado en θ_0, normalmente con pocos pasos y una tasa de aprendizaje pequeña.

En los modelos de lenguaje, la tarea de origen es la predicción del token siguiente sobre un corpus masivo (módulos 15 y 16), y θ_0 define un *modelo base*.

Definición 1.2 (Modelo base). Un *modelo base* es un modelo de lenguaje autorregresivo entrenado únicamente con la entropía cruzada del token siguiente sobre texto general. Para un contexto $x_{1:t-1}$, devuelve una aproximación de la distribución condicional del corpus, $p_{\theta_0}(x_t \mid x_{1:t-1})$.

Observación 1.4. Como el problema de destino no es convexo y la optimización es corta, el resultado depende de θ_0 no solo como punto de partida: el método iterativo permanece cerca de θ_0 y actúa como una regularización implícita. El tema 3 hace explícita esta regularización mediante una penalización de la distancia a θ_0.

1.2 Extracción de características y ajuste completo

Sea $\phi_\theta(x) \in \mathbb{R}^h$ la representación que la red asigna a una entrada (por ejemplo, el vector oculto de la última capa) y sea una *cabeza* lineal $z = V\phi_\theta(x) + b$, con $V \in \mathbb{R}^{K \times h}$, que produce los logits de K clases.

Definición 1.5 (Extracción de características y ajuste completo). En la *extracción de características* (o *sonda lineal*) se fijan los parámetros del cuerpo en $\theta = \theta_0$ y solo se entrenan (V, b). En el *ajuste completo* se entrenan a la vez θ, inicializado en θ_0, y la cabeza, inicializada al azar.

Proposición 1.6 (La sonda lineal es convexa). *Con el cuerpo congelado, la pérdida de entropía cruzada media*

$$J(V,b)=\frac{1}{n}\sum_{i=1}^{n}\Big(\mathrm{LSE}\big(V\phi_{\theta_0}(x_i)+b\big)-\big(V\phi_{\theta_0}(x_i)+b\big)_{y_i}\Big)$$

es una función convexa de (V,b).

En el ajuste completo, ϕ_θ depende de θ de forma no lineal y la convexidad se pierde. A cambio, el cuerpo puede adaptar sus representaciones a la tarea. La receta habitual [28, §14.2] usa una tasa de aprendizaje pequeña en el cuerpo, que ya está cerca de una buena solución, y una mayor (por ejemplo, diez veces mayor) en la cabeza, que parte de cero.

1.3 ULMFiT: ajuste fino de un modelo de lenguaje

Howard y Ruder [17] propusieron en 2018 el primer procedimiento general para ajustar un modelo de lenguaje preentrenado a tareas de clasificación de textos. Consta de tres etapas: preentrenamiento del modelo de lenguaje en un corpus general, ajuste del modelo de lenguaje al texto del dominio de destino y ajuste del clasificador. Introduce tres técnicas para que el ajuste no destruya lo aprendido.

Definición 1.7 (Tasas de aprendizaje discriminativas). Sea una red de L capas con parámetros $\theta^1,\dots,\theta^L$. El *ajuste discriminativo* asigna a cada capa su propia tasa, $\theta^l_t=\theta^l_{t-1}-\eta^l\nabla_{\theta^l}J$, con $\eta^{l-1}=\eta^l/2{,}6$: cada capa aprende 2,6 veces más despacio que la siguiente. La tasa η^L de la última capa se elige ajustando solo esa capa [17, §3.2].

Por inducción, $\eta^l=\eta^L/2{,}6^{\,L-l}$. La justificación es que las capas inferiores capturan información más general, que conviene modificar menos.

Definición 1.8 (Tasa triangular inclinada). Sean T el número total de iteraciones, $\mathrm{cf}\in(0,1)$ la fracción de calentamiento, $\rho>1$ el cociente entre la tasa máxima y la mínima y $\eta_{\max}$ la tasa máxima. Con $c=\lfloor T\,\mathrm{cf}\rfloor$, la *tasa triangular inclinada* (STLR) en la iteración t es

$$p_t=\begin{cases}t/c, & t<c,\\ 1-\dfrac{t-c}{c\,(1/\mathrm{cf}-1)}, & t\geq c,\end{cases}\qquad \eta_t=\eta_{\max}\,\frac{1+p_t(\rho-1)}{\rho}.$$

Los valores por defecto del artículo son $\mathrm{cf}=0{,}1$, $\rho=32$ y $\eta_{\max}=0{,}01$ [17, §3.2].

Proposición 1.9 (Forma de la tasa triangular). *Supóngase que* $T\,\mathrm{cf}$ *es entero, de modo que* $c=T\,\mathrm{cf}$. *Entonces:*

(i) η_t *es una función afín de* p_t, *creciente, con* $\eta=\eta_{\max}/\rho$ *si* $p_t=0$ *y* $\eta=\eta_{\max}$ *si* $p_t=1$;

(ii) p_t *crece linealmente de* 0 *a* 1 *en* $[0,c]$ *y decrece linealmente de* 1 *a* 0 *en* $[c,T]$;

(iii) en consecuencia, la tasa sube en c *iteraciones desde* $\eta_{\max}/\rho$ *hasta* $\eta_{\max}$ *y baja en* $T-c$ *iteraciones hasta* $\eta_{\max}/\rho$.

Definición 1.10 (Descongelación gradual). En la *descongelación gradual* se entrena primero solo la última capa durante una época; después se descongela la capa anterior y se entrenan todas las descongeladas otra época, y así sucesivamente hasta entrenar todas las capas [17, §3.3].

Observación 1.12. Las tres técnicas persiguen el mismo fin: evitar que los primeros pasos del ajuste, con gradientes grandes procedentes de una cabeza aleatoria, destruyan las representaciones del modelo preentrenado. Howard y Ruder las presentan como remedio al *olvido catastrófico*, que el tema 3 estudia con detalle. La tasa triangular es, además, un antecedente directo del calentamiento seguido de decaimiento que se usa en el entrenamiento de los Transformer (módulo 7).

1.4 Del ajuste por tareas al posentrenamiento

ULMFiT añade una cabeza de clasificación y produce un modelo para una sola tarea. En los modelos de lenguaje generativos se conserva la cabeza de lenguaje: la tarea de destino se formula también como predicción del token siguiente, pero sobre textos que tienen la forma de una conversación entre un usuario y un asistente. Jurafsky y Martin [19, cap. 8] llaman *posentrenamiento* al conjunto de etapas que siguen al preentrenamiento y lo dividen en el ajuste con instrucciones (§8.1, tema 2) y el aprendizaje a partir de preferencias (§8.3 y §8.4, módulo 21). La receta en tres pasos de InstructGPT [25] (ajuste supervisado, modelo de recompensa y aprendizaje por refuerzo) es la referencia publicada que siguen casi todos los asistentes (figura 1.1).

Observación 1.13 (Información pública sobre modelos comerciales). Anthropic, Google y xAI no publican los datos ni los hiperparámetros del ajuste supervisado de Claude, Gemini y Grok. Lo público es el esquema general. Anthropic describió en 2022 el método *Constitutional AI* [2], que incluye una fase supervisada en la que el modelo critica y revisa sus propias respuestas según una lista de principios y se ajusta sobre las respuestas revisadas, seguida de aprendizaje por refuerzo con retroalimentación de un modelo (módulo 21). El informe técnico de Gemini [13, §6.3] describe, a grandes rasgos, un ajuste supervisado seguido de aprendizaje por refuerzo con retroalimentación humana. xAI no ha publicado una descripción equivalente del posentrenamiento de Grok.

Tema 2

Ajuste supervisado con instrucciones

Este tema define el ajuste supervisado (SFT, del inglés *supervised fine-tuning*) de un modelo de lenguaje: la misma entropía cruzada del preentrenamiento, evaluada solo en las posiciones de la respuesta. Se estudian los datos de instrucciones, la pérdida enmascarada y su interpretación probabilística, los detalles de construcción de lotes y la plantilla de chat. Se sigue el §8.1 de Jurafsky y Martin [19], la clase 15 de CS336 [6] y su tarea 5 [8, §4.2], y los artículos de FLAN [27], T0 [26], InstructGPT [25, §3.2 y §3.5] y LIMA [29].

2.1 Datos de instrucciones

Definición 2.1 (Conjunto de instrucciones). Un *conjunto de instrucciones* es una colección de pares $\{(u^{(i)}, y^{(i)})\}_{i=1}^{n}$ en la que $u^{(i)}$ es una instrucción en lenguaje natural, que puede incluir un texto de entrada, e $y^{(i)}$ es la respuesta deseada. El *ajuste con instrucciones* (*instruction tuning*) es el ajuste supervisado sobre un conjunto de instrucciones que abarca muchas tareas distintas.

Los conjuntos se construyen de dos maneras [19, §8.1.1]: reformulando como instrucciones conjuntos de datos supervisados ya existentes, mediante plantillas, o recogiendo demostraciones escritas por personas (o generadas por otros modelos). La colección Flan 2022, por ejemplo, reúne 15 millones de ejemplos de 1836 tareas [19, §8.1.1].

Definición 2.2 (Generalización a tareas no vistas). Se agrupan las tareas en *grupos* (traducción, resumen, inferencia, preguntas y respuestas, etc.). Para medir la generalización a tareas no vistas se ajusta el modelo con todos los grupos salvo uno y se evalúa sin ejemplos (*zero-shot*) en las tareas del grupo excluido.

Con este protocolo, Wei *et al.* [27] ajustaron un modelo de 137 000 millones de parámetros con más de 60 conjuntos de datos expresados mediante plantillas de instrucciones (FLAN). El modelo ajustado superó, sin ejemplos, a GPT-3 de 175 000 millones de parámetros en 20 de los 25 conjuntos evaluados, y el beneficio dependía de la escala: en los modelos más pequeños estudiados, el ajuste con instrucciones empeoraba el resultado en las tareas no vistas. Sanh *et al.* [26] obtuvieron resultados análogos con T0, un modelo de tipo T5 de 11 000 millones de parámetros ajustado con una colección de conjuntos de datos reformulados mediante múltiples *prompts*, que igualaba o superaba en varias pruebas a modelos hasta 16 veces mayores.

Observación 2.3 (El ajuste supervisado en InstructGPT). En InstructGPT [25], el primer paso usa unas 13 000 instrucciones de entrenamiento con demostraciones escritas por anotadores [25, §3.2]. El modelo se ajusta durante 16 épocas con decaimiento coseno de la tasa de aprendizaje y *dropout* residual de 0,2; la pérdida de validación empieza a subir después de la primera época,

pero entrenar más épocas mejora la puntuación del modelo de recompensa y las preferencias humanas [25, §3.5]. La pérdida de validación del SFT no es, por tanto, un indicador fiable de la calidad final.

2.2 La pérdida enmascarada

Sean $u = u_{1:n}$ la instrucción e $y = y_{1:m}$ la respuesta, ambas ya tokenizadas, y sea $x_{1:T} = (u_1, \dots, u_n, y_1, \dots, y_m)$, con $T = n + m$, la secuencia concatenada. El último token de y es el token de fin de turno.

Definición 2.4 (Pérdida del ajuste supervisado). Sea $m_t \in \{0, 1\}$ una *máscara* que vale 1 en las posiciones de la respuesta ($t > n$) y 0 en las de la instrucción. La *pérdida del ajuste supervisado* es la entropía cruzada enmascarada

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{\sum_{t=1}^{T} m_t} \sum_{t=1}^{T} m_t \ln p_\theta(x_t \mid x_{1:t-1}).$$

Se divide por el número de posiciones que cuentan, no por T. Si $m_t = 1$ para todo t, se recupera la pérdida del preentrenamiento.

Proposición 2.5 (Interpretación probabilística). *Con la máscara de la definición 2.4,*

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{m} \ln p_\theta(y \mid u), \qquad \text{donde} \quad p_\theta(y \mid u) = \prod_{j=1}^{m} p_\theta(y_j \mid u, y_{1:j-1}).$$

En consecuencia, minimizar la suma de $m^{(i)}\mathcal{L}^{(i)}_{\mathrm{SFT}}$ sobre un conjunto de instrucciones equivale a estimar por máxima verosimilitud la distribución condicional de la respuesta dada la instrucción.

Observación 2.6. Sin máscara se minimizaría $-\frac{1}{T}\ln p_\theta(u, y) = -\frac{1}{T}\big(\ln p_\theta(u) + \ln p_\theta(y \mid u)\big)$: el modelo dedicaría parte de su capacidad a imitar las instrucciones de los usuarios, que no tiene que generar. Este es el sentido de la indicación de Jurafsky y Martin de entrenar solo sobre la respuesta [19, §8.1].

Proposición 2.7 (Gradiente en las posiciones enmascaradas). *Sea $z_t \in \mathbb{R}^K$ el vector de logits de la posición t y $p_t = \mathrm{softmax}(z_t)$. Entonces*

$$\frac{\partial \mathcal{L}_{\mathrm{SFT}}}{\partial z_t} = \frac{m_t}{\sum_s m_s}\,\big(p_t - e_{x_t}\big),$$

donde e_{x_t} es el vector canónico del token objetivo. En particular, el gradiente respecto a los logits de una posición enmascarada es nulo.

Observación 2.8. Que el gradiente respecto a los logits de la instrucción sea nulo no implica que la instrucción no influya en el aprendizaje: las posiciones de la respuesta atienden a las de la instrucción, de modo que el gradiente llega a los estados ocultos de la instrucción a través de la atención. Lo que se suprime es la señal que obligaría a *predecir* los tokens de la instrucción.

Observación 2.10 (Implementación). En PyTorch no se multiplica por la máscara de forma explícita: se asigna la etiqueta -100 a las posiciones que no cuentan y la función `cross_entropy` con `ignore_index=-100`, el valor por defecto, las excluye del numerador y del denominador de la media.

2.3 Lotes: relleno, normalización, empaquetado y acumulación

Las secuencias de un lote tienen longitudes distintas. Hay dos formas de formar tensores rectangulares.

Definición 2.11 (Relleno y empaquetado). En el *relleno* (*padding*) cada secuencia se completa con un token de relleno hasta la longitud máxima del lote, y la máscara vale 0 en las posiciones de relleno. En el *empaquetado* (*packing*) se concatenan los ejemplos, separados por el token de fin, y la cadena resultante se corta en trozos consecutivos de longitud fija L.

El empaquetado no desperdicia posiciones, pero un trozo puede contener el final de un ejemplo y el principio de otro; la máscara causal permite entonces que un ejemplo atienda al anterior, lo que suele tolerarse porque el token de fin los separa.

Sea un lote de B secuencias con máscaras $m_t^{(i)}$ y pérdidas por token $\ell_t^{(i)} = -\ln p_\theta(x_t^{(i)} \mid x_{1:t-1}^{(i)})$, y sea $M_i = \sum_t m_t^{(i)}$. Hay dos normalizaciones habituales:

$$J_{\text{tok}} = \frac{\sum_i \sum_t m_t^{(i)} \ell_t^{(i)}}{\sum_i M_i}, \qquad J_{\text{sec}} = \frac{1}{B} \sum_{i=1}^{B} \frac{1}{c} \sum_t m_t^{(i)} \ell_t^{(i)},$$

con $c > 0$ una constante fija (por ejemplo, la longitud máxima). En J_{tok} cada secuencia pesa en proporción a la longitud de su respuesta; en J_{sec} todas las secuencias pesan lo mismo por cada token, independientemente del lote en el que caigan. La tarea 5 de CS336 implementa una normalización del segundo tipo: una suma enmascarada dividida por una constante [8, §4.2].

Proposición 2.12 (Acumulación de gradientes). *Divídase un lote de B secuencias en G microlotes $\mathcal{B}_1, \dots, \mathcal{B}_G$ de B/G secuencias. Sea J_g la pérdida J_{sec} del microlote g calculada con tamaño B/G. Si cada microlote retropropaga J_g/G y se suman los gradientes antes de actualizar, el gradiente acumulado coincide con el de J_{sec} sobre el lote completo.*

Observación 2.13. Con la normalización J_{tok} la equivalencia falla en general: cada microlote divide por su propio número de tokens de respuesta y la suma de los cocientes no es el cociente de las sumas, salvo que todos los microlotes tengan el mismo número de tokens que puntúan.

2.4 La plantilla de chat

Un diálogo es una lista de mensajes (r_1, c_1), …, (r_J, c_J), donde $r_j \in \{\text{sistema}, \text{usuario}, \text{asistente}\}$ es el rol y c_j el contenido. Para tratarlo con un modelo autorregresivo hay que convertirlo en una única secuencia.

Definición 2.14 (Plantilla de chat). Sea $\mathcal{V}$ un vocabulario ampliado con un token especial por rol, $\langle r \rangle$, y un token de fin de turno, $\langle \text{fin} \rangle$, que no aparecen en el texto ordinario. La *plantilla de chat* asigna al diálogo la secuencia

$$\langle r_1 \rangle\, \tau(c_1)\, \langle \text{fin} \rangle\ \langle r_2 \rangle\, \tau(c_2)\, \langle \text{fin} \rangle\ \cdots\ \langle r_J \rangle\, \tau(c_J)\, \langle \text{fin} \rangle,$$

donde τ es el tokenizador. En el ajuste supervisado, la máscara vale 1 en los tokens de $\tau(c_j)$ y en el $\langle \text{fin} \rangle$ que los cierra si $r_j = \text{asistente}$, y 0 en el resto.

En la inferencia, la aplicación construye la plantilla con todo el historial, añade $\langle \text{asistente} \rangle$ y deja que el modelo genere hasta emitir $\langle \text{fin} \rangle$. Incluir el token de fin en la pérdida es imprescindible para que el modelo aprenda a terminar su turno: en la práctica del módulo, el modelo

base no lo emite nunca y, tras 600 pasos de SFT, lo emite en el 99,4 % de los ejemplos de entrenamiento y en el 100 % de los de prueba.

Observación 2.15. Los tokens especiales se añaden al vocabulario antes del ajuste (módulo 12) y sus vectores de *embedding* se aprenden durante él. Por eso la plantilla forma parte del modelo: usar un modelo ajustado con una plantilla distinta de la suya degrada su comportamiento. Cada laboratorio usa su propio formato interno y no siempre lo publica; lo que documentan públicamente las API de Claude, Gemini y Grok es que los mensajes se envían como una lista de pares (rol, contenido).

2.5 ¿Cuántos datos hacen falta? La alineación superficial

El preentrenamiento consume billones de tokens; el ajuste supervisado publicado usa entre miles y cientos de miles de diálogos. Zhou *et al.* [29] ajustaron un modelo LLaMa de 65 000 millones de parámetros con solo 1000 pares instrucción–respuesta seleccionados con cuidado (LIMA); en una evaluación humana controlada, sus respuestas empataron con las de GPT-4 o fueron preferidas a ellas en el 43 % de los casos. De ahí la *hipótesis de la alineación superficial*: casi todo el conocimiento del modelo se adquiere en el preentrenamiento, y el ajuste enseña sobre todo qué formato y qué estilo usar al interactuar con un usuario. En la misma línea, Dettmers *et al.* [11, §1] observaron que un conjunto de 9000 diálogos de alta calidad (OASST1) daba mejores asistentes que una submuestra de 450 000 ejemplos de FLAN v2.

Observación 2.16. Estas conclusiones proceden de evaluaciones concretas de calidad conversacional y no implican que los datos de SFT sean irrelevantes para todas las capacidades: tareas como el razonamiento matemático o la programación se benefician de muchos más ejemplos. La lección robusta es que la calidad y la diversidad de los datos pesan más que su cantidad.

Tema 3

Olvido catastrófico

Al ajustar todos los pesos con una tarea estrecha, el modelo pierde capacidades que tenía. Este tema define el fenómeno, lo analiza en un modelo cuadrático que puede resolverse a mano y deduce la penalización de la consolidación elástica de pesos (EWC) como aproximación bayesiana. El fenómeno se describió en redes conexionistas en el trabajo de McCloskey y Cohen [24]; EWC procede de Kirkpatrick *et al.* [20]; la aproximación de Laplace sigue el §4.4 de Bishop [3], y el análisis cuadrático de la penalización, el §7.1.1 de Goodfellow, Bengio y Courville [14].

3.1 El fenómeno

Definición 3.1 (Olvido). Sean $\mathcal{L}_A$ la pérdida de una tarea antigua, θ_0 unos parámetros que la resuelven y θ_B los parámetros obtenidos al ajustar θ_0 con una tarea nueva de pérdida $\mathcal{L}_B$. El *olvido* es el aumento $\mathcal{L}_A(\theta_B) - \mathcal{L}_A(\theta_0)$. Se habla de *olvido catastrófico* cuando ese aumento es comparable al que tendría un modelo sin entrenar en la tarea A.

McCloskey y Cohen [24] mostraron en 1989 que una red entrenada secuencialmente en dos tareas pierde casi por completo la primera al aprender la segunda, a diferencia de lo que ocurre en el aprendizaje humano. La causa es que los mismos parámetros codifican ambas tareas y el gradiente de $\mathcal{L}_B$ no tiene en cuenta cuánto importa cada parámetro para A.

3.2 Un modelo cuadrático del compromiso

Proposición 3.3 (Muelle hacia el modelo preentrenado). *Sean $a, \theta_0 \in \mathbb{R}$ y $\lambda \geq 0$, y sea*

$$\mathcal{L}(\theta) = (\theta - a)^2 + \lambda(\theta - \theta_0)^2.$$

El único minimizador es $\theta^* = (a + \lambda\theta_0)/(1 + \lambda)$. En él, el olvido medido como $(\theta^* - \theta_0)^2$ y el error en la tarea nueva $(\theta^* - a)^2$ valen

$$(\theta^* - \theta_0)^2 = \frac{(a - \theta_0)^2}{(1 + \lambda)^2}, \qquad (\theta^* - a)^2 = \frac{\lambda^2 (a - \theta_0)^2}{(1 + \lambda)^2}.$$

Corolario 3.4. *Las distancias $|\theta^* - \theta_0|$ y $|\theta^* - a|$ suman $|a - \theta_0|$ para todo $\lambda \geq 0$: θ^* recorre el segmento entre a y θ_0, desde $\theta^* = a$ ($\lambda = 0$, olvido máximo) hasta $\theta^* \to \theta_0$ ($\lambda \to \infty$, ningún aprendizaje).*

La penalización $\lambda(\theta - \theta_0)^2$ es la energía potencial de un muelle de constante 2λ anclado en θ_0, y θ^* es el punto de equilibrio entre dos fuerzas lineales. El problema real tiene millones de parámetros, y no todos importan igual para la tarea antigua: conviene un muelle más rígido en los parámetros importantes. Esa es la idea de EWC.

3.3 Consolidación elástica de pesos

3.3.1 Justificación bayesiana

Sean $\mathcal{D}_A$ y $\mathcal{D}_B$ los datos de las dos tareas, condicionalmente independientes dado θ. Por el teorema de Bayes,

$$\ln p(\theta \mid \mathcal{D}_A, \mathcal{D}_B) = \ln p(\mathcal{D}_B \mid \theta) + \ln p(\theta \mid \mathcal{D}_A) - \ln p(\mathcal{D}_B \mid \mathcal{D}_A). \tag{3.1}$$

El último término no depende de θ. Toda la información sobre la tarea A está en la distribución a posteriori $p(\theta \mid \mathcal{D}_A)$, que no puede calcularse exactamente. La *aproximación de Laplace* [3, §4.4] la sustituye por una gaussiana centrada en su moda.

Definición 3.6 (Aproximación de Laplace). Sea θ_A un máximo de $g(\theta) = \ln p(\theta \mid \mathcal{D}_A)$ y sea $H_A = -\nabla^2 g(\theta_A)$, definida positiva. La aproximación de Laplace es $p(\theta \mid \mathcal{D}_A) \approx \mathcal{N}(\theta; \theta_A, H_A^{-1})$, que se obtiene del desarrollo de Taylor de segundo orden $g(\theta) \approx g(\theta_A) - \frac{1}{2}(\theta - \theta_A)^\top H_A (\theta - \theta_A)$, en el que el término lineal se anula porque $\nabla g(\theta_A) = 0$.

Sustituyendo en (3.1), maximizar la distribución a posteriori conjunta equivale aproximadamente a minimizar

$$-\ln p(\mathcal{D}_B \mid \theta) + \tfrac{1}{2}(\theta - \theta_A)^\top H_A (\theta - \theta_A).$$

Queda por estimar H_A, una matriz de $D \times D$ con D del orden de millones.

3.3.2 Información de Fisher

Definición 3.7 (Información de Fisher). Sea p_θ una familia de distribuciones derivable en θ. Su *matriz de información de Fisher* es

$$\mathcal{I}(\theta) = \mathbb{E}_{x \sim p_\theta}\left[\nabla_\theta \ln p_\theta(x)\, \nabla_\theta \ln p_\theta(x)^\top\right].$$

El vector $\nabla_\theta \ln p_\theta(x)$ se llama *función de puntuación* (*score*).

$\mathcal{I}(\theta)$ es semidefinida positiva por ser la esperanza de matrices de la forma $vv^\top$. Solo requiere gradientes de primer orden.

Proposición 3.8 (Fisher como hessiano esperado). *Supóngase que se pueden intercambiar derivada e integral. Entonces* $\mathbb{E}_{x \sim p_\theta}[\nabla_\theta \ln p_\theta(x)] = 0$ *y*

$$\mathcal{I}(\theta) = -\mathbb{E}_{x \sim p_\theta}\left[\nabla^2_\theta \ln p_\theta(x)\right].$$

Si $\mathcal{D}_A$ consta de n ejemplos y la distribución a priori es plana, $H_A = -\sum_i \nabla^2 \ln p_\theta(x_i)\big|_{\theta_A}$, y por la ley de los grandes números y la proposición 3.8 se aproxima por $n\,\mathcal{I}(\theta_A)$ cuando el modelo describe bien los datos. Kirkpatrick *et al.* se quedan además con la diagonal, $F_i = \mathcal{I}_{ii}(\theta_A)$, que se estima promediando los cuadrados de las componentes del gradiente.

Definición 3.9 (EWC). Sean θ_A los parámetros tras la tarea A y F_i la diagonal de su matriz de Fisher. La *consolidación elástica de pesos* minimiza

$$\mathcal{L}_{\mathrm{EWC}}(\theta) = \mathcal{L}_B(\theta) + \sum_{i=1}^{D} \frac{\lambda}{2} F_i \left(\theta_i - \theta_{A,i}\right)^2,$$

donde $\lambda > 0$ regula la importancia de la tarea antigua [20].

Cada parámetro tiene su propio muelle, de constante λF_i: los parámetros a los que la verosimilitud de la tarea A es sensible (Fisher grande) quedan rígidos y los demás pueden moverse libremente. La proposición 3.3 es el caso de un solo parámetro en el que el coeficiente $\frac{\lambda}{2}F$ de EWC hace el papel del λ de la proposición.

3.4 Otros remedios

Definición 3.10 (Repetición). La *repetición* (*replay*) mezcla en cada paso ejemplos de la tarea antigua: con $\beta \in [0, 1)$, se minimiza $(1-\beta)\mathcal{L}_B(\theta) + \beta\,\mathcal{L}_A(\theta)$.

Otros remedios habituales son una tasa de aprendizaje pequeña y pocas épocas, las técnicas de ULMFiT (tema 1) y modificar pocos parámetros, como hacen los métodos del tema 4 y LoRA (tema 5). El módulo 21 penalizará la divergencia de Kullback–Leibler entre el modelo ajustado y uno de referencia. Esa penalización actúa en el espacio de las distribuciones de salida, pero localmente coincide con un muelle en el espacio de parámetros cuya rigidez es la información de Fisher.

Proposición 3.11 (Desarrollo local de la divergencia KL). *Bajo las hipótesis de la proposición 3.8, y si* $\ln p_\theta$ *es tres veces derivable,*

$$D_{\mathrm{KL}}\big(p_{\theta_0} \,\big\|\, p_{\theta_0+\delta}\big) = \tfrac{1}{2}\,\delta^\top \mathcal{I}(\theta_0)\,\delta + O(\|\delta\|^3).$$

Tema 4

Ajuste eficiente en parámetros

El ajuste completo de un modelo de D parámetros produce, para cada tarea, una copia de D parámetros y exige guardar durante el entrenamiento el estado del optimizador de todos ellos. Los métodos de *ajuste eficiente en parámetros* (PEFT, del inglés *parameter-efficient fine-tuning*) congelan el modelo preentrenado y entrenan un número pequeño de parámetros nuevos. Este tema presenta los adaptadores, el ajuste de *prompt* y de prefijo, y la noción de dimensión intrínseca que explica por qué bastan tan pocos parámetros. Se sigue el §8.2 de Jurafsky y Martin [19], la clase 9 de CS224N [5] y los artículos originales [16, 21, 23, 22, 1]; LoRA, el método más usado, ocupa el tema 5.

4.1 Planteamiento

Definición 4.1 (Ajuste eficiente en parámetros). Sea $\theta_0 \in \mathbb{R}^D$ un modelo preentrenado. Un método PEFT define una familia de modelos $f_{\theta_0,\varphi}$ con parámetros nuevos $\varphi \in \mathbb{R}^P$, $P \ll D$, tal que existe φ_0 con $f_{\theta_0,\varphi_0} = f_{\theta_0}$, y entrena solo φ, partiendo de φ_0 o de un valor próximo, con θ_0 congelado.

Los métodos se agrupan en tres familias: *aditivos*, que añaden módulos o vectores nuevos (adaptadores, ajuste de *prompt* y de prefijo); *selectivos*, que entrenan un subconjunto de los parámetros existentes (la sonda lineal del tema 1 es el caso extremo), y de *reparametrización*, que escriben la actualización $\Delta\theta$ como función de pocos parámetros (subespacios aleatorios, apartado 4.4, y LoRA, tema 5).

Proposición 4.2 (Memoria del entrenamiento). *Con precisión mixta y Adam, el ajuste completo requiere unos* $16D$ *bytes para pesos, gradientes y estado del optimizador (módulo 17). Con un método PEFT y el modelo base en bf16, la memoria es de unos* $2D + 16P$ *bytes, sin contar las activaciones.*

Con $D = 7 \cdot 10^9$ y $P = 2 \cdot 10^7$, la memoria baja de 112 GB a $14 + 0{,}32 = 14{,}32$ GB. Además, cada tarea se almacena con P parámetros en lugar de D, y un mismo modelo base puede servir muchas tareas cambiando solo φ.

4.2 Adaptadores

Definición 4.3 (Adaptador). Sea $h \in \mathbb{R}^d$ la salida de una subcapa de un Transformer. Un *adaptador* con dimensión de cuello $m \ll d$ calcula

$$a(h) = h + W_{\uparrow}\, g\big(W_{\downarrow} h + b_{\downarrow}\big) + b_{\uparrow}, \qquad W_{\downarrow} \in \mathbb{R}^{m \times d},\ W_{\uparrow} \in \mathbb{R}^{d \times m},$$

con g una no linealidad. Houlsby *et al.* [16] insertan dos adaptadores por capa, tras la subcapa de atención y tras la red de alimentación hacia delante, y entrenan además los parámetros de

normalización de capa.

Proposición 4.4. *Un adaptador tiene $2md + m + d$ parámetros. Si se inicializa con $W_\uparrow = 0$ y $b_\uparrow = 0$, entonces $a(h) = h$ para todo h, de modo que el modelo con adaptadores coincide al principio con el preentrenado.*

Houlsby *et al.* usan una inicialización *casi* nula, de modo que el adaptador es casi la identidad, y obtienen en GLUE resultados a menos de un 0,4 % de los del ajuste completo añadiendo un 3,6 % de parámetros por tarea [16].

Observación 4.5 (Latencia). Los adaptadores añaden capas que se ejecutan en serie y no pueden fusionarse con los pesos originales, porque contienen una no linealidad. Hu *et al.* [18, §3] midieron aumentos de la latencia de inferencia superiores al 30 % con secuencias cortas y lotes pequeños. LoRA evita este problema (tema 5).

4.3 Ajuste de *prompt* y de prefijo

Definición 4.6 (Ajuste de *prompt*). Sea $E \in \mathbb{R}^{T\times d}$ la matriz de *embeddings* de una entrada. El *ajuste de prompt* antepone p vectores entrenables $P_e \in \mathbb{R}^{p\times d}$ y alimenta el modelo congelado con $[P_e; E] \in \mathbb{R}^{(p+T)\times d}$. Tiene pd parámetros [21].

Los vectores de P_e no tienen por qué corresponder a ningún token del vocabulario: forman un *prompt continuo* (*soft prompt*). Lester *et al.* [21] observaron que la diferencia con el ajuste completo disminuye con la escala y prácticamente desaparece en modelos de unos 10^{10} parámetros.

Definición 4.7 (Ajuste de prefijo). El *ajuste de prefijo* antepone, en cada una de las L capas, p claves y p valores entrenables, $P_K^l, P_V^l \in \mathbb{R}^{p\times d}$, a las claves y valores calculados a partir de la entrada. Tiene $2Lpd$ parámetros. Para estabilizar el entrenamiento, Li y Liang [23] reparametrizan los prefijos mediante una pequeña red que se descarta al terminar; con ello entrenan en torno al 0,1 % de los parámetros.

El efecto de un prefijo sobre la atención admite una descripción exacta.

Teorema 4.8 (El prefijo interpola la atención). *Sean $q \in \mathbb{R}^d$ una consulta, $K, V \in \mathbb{R}^{T\times d}$ las claves y valores de la entrada y $P_K, P_V \in \mathbb{R}^{p\times d}$ los del prefijo. Escríbase $\mathrm{At}(q, K, V) = \mathrm{softmax}(Kq/\sqrt{d})^\top V$. Entonces*

$$\mathrm{At}\big(q, [P_K; K], [P_V; V]\big) = \big(1 - \mu(q)\big)\,\mathrm{At}(q, K, V) + \mu(q)\,\mathrm{At}(q, P_K, P_V),$$

con

$$\mu(q) = \frac{S_P}{S_P + S_K}, \qquad S_P = \sum_{j=1}^{p} e^{(P_K)_j \cdot q/\sqrt{d}}, \quad S_K = \sum_{i=1}^{T} e^{k_i \cdot q/\sqrt{d}}.$$

Corolario 4.9. *El ajuste de prefijo no altera los pesos relativos de atención entre las posiciones de la entrada: solo mezcla la salida original con un vector que depende de la consulta, con una proporción $\mu(q) \in (0, 1)$.*

Esta lectura, que relaciona el ajuste de prefijo con los adaptadores, se desarrolla en He *et al.* [15].

4.4 Dimensión intrínseca

¿Por qué basta con ajustar una fracción mínima de los parámetros? Li *et al.* [22] propusieron medir la dimensión efectiva de un problema de optimización restringiendo el entrenamiento a un subespacio aleatorio.

Definición 4.10 (Subespacio aleatorio y dimensión intrínseca). Sean $\theta_0 \in \mathbb{R}^D$ y $P \in \mathbb{R}^{D\times d'}$ una matriz aleatoria fija. Se entrena solo $z \in \mathbb{R}^{d'}$, inicializado en 0, con la reparametrización $\theta = \theta_0 + Pz$. La *dimensión intrínseca* d_{90} de una tarea es el menor d' con el que se alcanza el 90 % del rendimiento del ajuste completo [1, §3].

Proposición 4.11 (Gradiente en el subespacio). *Si $\mathcal{L}$ es derivable,* $\nabla_z \mathcal{L}(\theta_0 + Pz) = P^\top \nabla_\theta \mathcal{L}(\theta)\big|_{\theta=\theta_0+Pz}$.

Aghajanyan *et al.* [1] estimaron la dimensión intrínseca del ajuste de modelos de lenguaje preentrenados, usando para P la transformada Fastfood, que evita almacenar una matriz densa de $D \times d'$. Sus tres resultados principales son:

(i) con RoBERTa-Large, unos 200 parámetros bastan para alcanzar el 90 % del rendimiento del ajuste completo en la tarea MRPC [1, §4.2];

(ii) la dimensión intrínseca disminuye a lo largo del preentrenamiento y es menor en los modelos con más parámetros [1, §5.1 y §5.2];

(iii) una cota de generalización basada en compresión depende de la dimensión intrínseca y no del número total de parámetros: con n ejemplos, el error de generalización es $O(\sqrt{d_{90}/n})$, para modelos entrenados en el subespacio [1, §5.3].

Observación 4.12. El segundo resultado explica en parte la paradoja de que los modelos mayores se ajusten con menos datos: el preentrenamiento comprime el problema de adaptación. Hu *et al.* formularon a partir de aquí la hipótesis que motiva LoRA [18, §1 y §4.1]: si el ajuste vive en un subespacio de dimensión pequeña, también la actualización de cada matriz de pesos debe tener un *rango intrínseco* bajo. LoRA no fija las direcciones al azar: aprende la mejor base de rango r.

Tema 5

LoRA: adaptación de bajo rango

LoRA (*low-rank adaptation*) congela cada matriz preentrenada y aprende solo una corrección de rango bajo, escrita como producto de dos matrices delgadas. Es hoy el método PEFT más usado porque no añade latencia de inferencia y su análisis se reduce al álgebra lineal del módulo 3. Este tema sigue el artículo original de Hu *et al.* [18, §4 y §7], el teorema de Eckart y Young [12] en la forma del módulo 3 y de Deisenroth, Faisal y Ong [10, §4.6], y la clase invitada sobre LoRA del curso CS224N [5].

5.1 Definición y número de parámetros

Definición 5.1 (LoRA). Sea $W \in \mathbb{R}^{d\times k}$ una matriz preentrenada de una capa lineal $h = Wx$, con $x \in \mathbb{R}^k$. Sean $r \ll \min(d,k)$ el *rango*, $\alpha > 0$ un hiperparámetro de escala y $s = \alpha/r$. LoRA sustituye la capa por

$$h = Wx + s\,BAx, \qquad A \in \mathbb{R}^{r\times k},\ B \in \mathbb{R}^{d\times r},$$

con W congelada y A, B entrenables. Se inicializa A con entradas gaussianas y $B = 0$ [18, §4.1]. La matriz efectiva es $W' = W + s\,BA$.

Proposición 5.2 (Parámetros y coste). *LoRA entrena $r(d+k)$ parámetros por matriz, una fracción $r(d+k)/(dk) = r(1/d + 1/k)$ de los dk del ajuste completo. Calcular $s\,B(Ax)$ cuesta $2r(d+k)$ operaciones más un escalado, frente a las $2dk$ de Wx, y nunca requiere formar BA.*

Cuadro 5.1: Parámetros entrenables de LoRA en una matriz.

Matriz	r	Completo dk	LoRA $r(d+k)$	Fracción
64×64	4	4096	512	12,5 %
768×768	4	589 824	6144	1,04 %
4096×4096	8	16 777 216	65 536	0,39 %
4096×4096	16	16 777 216	131 072	0,78 %

En el mini-GPT de la práctica ($d = 64$, red de alimentación de dimensión 256, 2 bloques), LoRA con $r = 4$ en las cuatro matrices de atención (64×64) y las dos de la red de alimentación (64×256 y 256×64) de cada bloque entrena $2\cdot(4\cdot512+2\cdot1280) = 9216$ parámetros, el 8,57 % de los 107 520 del modelo. En GPT-3 de 175 000 millones de parámetros, Hu *et al.* redujeron los parámetros entrenables unas 10 000 veces y la memoria de GPU unas 3 veces respecto al ajuste completo con Adam; con $r = 4$ en W_Q y W_V, el punto de control pasó de unos 350 GB a unos 35 MB [18, §4.2].

Proposición 5.4 (Fusión). *Tras el entrenamiento, sustituir W por $W' = W + s\,BA$ produce la misma función que la capa LoRA y el mismo coste de inferencia que la capa original.*

Así, LoRA no añade latencia de inferencia, a diferencia de los adaptadores del tema 4. Sin fusionar, en cambio, un mismo modelo base puede servir muchos adaptadores pequeños e intercambiarlos en cada petición.

5.2 Rango de la corrección

Proposición 5.5 (Cota de rango). *Para todo $A \in \mathbb{R}^{r\times k}$ y $B \in \mathbb{R}^{d\times r}$,* $\operatorname{rango}(BA) \le r$.

Proposición 5.6 (Recíproco). *Toda matriz $M \in \mathbb{R}^{d\times k}$ con* $\operatorname{rango} M \le r$ *puede escribirse como $M = BA$ con $B \in \mathbb{R}^{d\times r}$ y $A \in \mathbb{R}^{r\times k}$.*

Corolario 5.7. *El conjunto de correcciones que LoRA de rango r puede representar es exactamente $\{\Delta W \in \mathbb{R}^{d\times k} : \operatorname{rango}\Delta W \le r\}$. Si $r < \min(d,k)$, ninguna corrección de rango completo es representable.*

5.3 Gradientes e inicialización

Proposición 5.8 (Gradientes de LoRA). *Sea $h = Wx + s\,BAx$ y sea $G = \partial\mathcal{L}/\partial h \in \mathbb{R}^d$ el gradiente que llega a la salida. Entonces*

$$\frac{\partial \mathcal{L}}{\partial B} = s\,G\,(Ax)^\top \in \mathbb{R}^{d\times r}, \qquad \frac{\partial \mathcal{L}}{\partial A} = s\,B^\top G\,x^\top \in \mathbb{R}^{r\times k}.$$

Con un lote, se suman las contribuciones de cada ejemplo.

Corolario 5.9 (Inicialización). *Con $B = 0$ y A aleatoria:*

(i) $BA = 0$ y el modelo ajustado empieza siendo exactamente el modelo base;

(ii) en el primer paso $\partial\mathcal{L}/\partial A = 0$, pero $\partial\mathcal{L}/\partial B = s\,G(Ax)^\top \neq 0$ si $G \neq 0$ y $Ax \neq 0$, de modo que B deja de ser nula y, desde el segundo paso, también A recibe gradiente.

En cambio, $(A,B) = (0,0)$ es un punto fijo del descenso de gradiente (ejercicio 5.7).

La elección simétrica, $A = 0$ con B aleatoria, también empieza en el modelo base y también permite aprender; el convenio del artículo original es $B = 0$.

5.4 La escala α/r

Proposición 5.10 (Adam es invariante a la escala del gradiente). *Sea el optimizador Adam con $\varepsilon = 0$ (módulo 7). Si se multiplican todos los gradientes por una constante $c > 0$, la sucesión de actualizaciones de los parámetros no cambia.*

Por la proposición 5.8, el gradiente de B es proporcional a s. Por la proposición 5.10, la actualización ΔB del primer paso no depende de s, y la variación de la función, $s\,\Delta B\,A\,x$, es proporcional a s. El factor $s = \alpha/r$ actúa, por tanto, como un multiplicador de la tasa de aprendizaje. Hu *et al.* observan que ajustar α con Adam equivale aproximadamente a ajustar la tasa, fijan α al primer valor de r que prueban y no lo modifican; dividir por r reduce la necesidad de reajustar hiperparámetros al cambiar el rango [18, §4.1]. En la práctica se fija α (por ejemplo, $\alpha = 16$ o $\alpha = 2r$) y se ajusta r.

5.5 Por qué funciona: relación con la SVD

LoRA apuesta a que la actualización ΔW que necesita el ajuste fino tiene un espectro que decae deprisa. El teorema de Eckart–Young (módulo 3, tema 5) cuantifica lo que se pierde al restringir el rango.

Teorema 5.11 (Eckart–Young, norma de Frobenius). *Sea $\Delta W = \sum_i \sigma_i u_i v_i^\top$ la SVD, con $\sigma_1 \geq \sigma_2 \geq \cdots$. Para todo r, la matriz $\Delta W_r = \sum_{i \leq r} \sigma_i u_i v_i^\top$ minimiza $\|\Delta W - M\|_F$ entre las matrices de rango a lo sumo r, y*

$$\|\Delta W - \Delta W_r\|_F = \Big(\sum_{i>r} \sigma_i^2\Big)^{1/2}.$$

La demostración está en el módulo 3 y en [12].

Corolario 5.12 (Error mínimo de LoRA). *Sea $\Delta W^\star$ la actualización ideal de una matriz. Entonces*

$$\min_{A,B} \|s\,BA - \Delta W^\star\|_F = \Big(\sum_{i>r} \sigma_i^2(\Delta W^\star)\Big)^{1/2}.$$

Hay dos diferencias con truncar una SVD. LoRA no calcula ninguna SVD: el descenso de gradiente encuentra B y A directamente. Y LoRA aproxima la *actualización*, no W, que suele tener rango completo y no se puede comprimir así sin perder mucho.

Observación 5.14 (Evidencia empírica). Hu *et al.* adaptan principalmente W_Q y W_V y encuentran que, en algunas tareas, un rango tan pequeño como $r = 1$ basta [18, §7.1 y §7.2]; las direcciones singulares principales aprendidas con $r = 8$ y con $r = 64$ se solapan mucho, y el resto parecen ruido. Dettmers *et al.* observaron, en cambio, que para igualar el ajuste completo en modelos grandes hace falta aplicar LoRA a todas las capas lineales [11, §4]. En la práctica del módulo, la actualización del ajuste completo en W_Q tiene rango 64 y sus 4 primeros valores singulares solo concentran el 41,5 % de la energía: un modelo tan pequeño, ajustado de forma tan agresiva, no cumple bien la hipótesis. Aun así, LoRA acierta más en la prueba (0,450 frente a 0,300, con tasas de aprendizaje distintas); una explicación plausible es que restringir el rango regulariza.

Tema 6

QLoRA y elección de estrategia

Con LoRA, la matriz W no recibe actualizaciones: solo se lee. Puede guardarse, por tanto, en muy baja precisión. QLoRA combina la cuantización del modelo base a 4 bits con adaptadores LoRA entrenados en 16 bits y permite ajustar modelos de decenas de miles de millones de parámetros en una sola GPU. Este tema estudia sus tres ingredientes (el formato NF4, la doble cuantización y los optimizadores paginados), hace la contabilidad de memoria de cada configuración y termina con los criterios para elegir entre ajuste fino, instrucciones en el contexto y recuperación de documentos. Se sigue el artículo de Dettmers *et al.* [11, §3 y §4], la contabilidad de parámetros de la tarea 1 de CS336 [7, §3.5] y el capítulo 8 de Jurafsky y Martin [19]; la cuantización por bloques se estudió en el módulo 19.

6.1 Cuantización por bloques

Definición 6.1 (Cuantización por bloques con un libro de códigos). Sea $\mathcal{Q} = \{q_0 < q_1 < \cdots < q_{2^b-1}\} \subset [-1, 1]$ un *libro de códigos* de 2^b niveles. Los pesos se dividen en bloques de B elementos. Para cada bloque $w \in \mathbb{R}^B$ se guarda la *constante de escala* $c = \max_j |w_j|$ y, para cada j, el índice del nivel de $\mathcal{Q}$ más próximo a w_j/c. La *descuantización* devuelve $\hat{w}_j = c\, q_{i_j}$.

Proposición 6.2 (Bits por parámetro). *Si cada índice ocupa b bits y cada constante se guarda en coma flotante de 32 bits, la cuantización por bloques ocupa $b + 32/B$ bits por parámetro.*

Con $b = 4$ y $B = 64$, el coste es $4 + 0{,}5 = 4{,}5$ bits por parámetro: la constante de escala añade un 12,5 % al tamaño de los índices.

6.2 El formato NF4

La elección del libro de códigos importa. Los pesos de una red preentrenada siguen aproximadamente una distribución normal de media nula, de modo que los valores próximos a cero son mucho más frecuentes que los extremos. Un libro de códigos uniforme desperdicia niveles en zonas poco pobladas.

Proposición 6.3 (Cuantización por cuantiles). *Sea $X \sim \mathcal{N}(0, 1)$ y sean $\beta_i = \Phi^{-1}(i/2^b)$, $i = 0, \ldots, 2^b$, con $\beta_0 = -\infty$ y $\beta_{2^b} = +\infty$. Si se asigna el código i a los valores de $[\beta_i, \beta_{i+1})$, cada código tiene probabilidad 2^{-b}, y la entropía del código es b bits, la máxima posible con 2^b símbolos.*

Así, cada uno de los 2^b códigos se usa con la misma frecuencia y ninguno se desperdicia. El *NormalFloat* de 4 bits (NF4) de Dettmers *et al.* aplica esta idea con $b = 4$: estima los niveles a partir de cuantiles de la normal estándar, los normaliza al intervalo $[-1, 1]$ y, para que el

cero sea representable exactamente, construye por separado 2^{b-1} niveles negativos y $2^{b-1}+1$ positivos y elimina el cero repetido [11, §3]. El tamaño de bloque es $B = 64$.

6.3 Doble cuantización

Proposición 6.5 (Coste de las constantes con doble cuantización). *Si las constantes de escala de primer nivel, una por bloque de B_1 pesos, se cuantizan a su vez a 8 bits en bloques de B_2 constantes, cada una con una constante de segundo nivel de 32 bits, el coste de las constantes es $8/B_1 + 32/(B_1B_2)$ bits por parámetro.*

Con $B_1 = 64$ y $B_2 = 256$, los valores de QLoRA [11, §3], el coste pasa de $32/64 = 0{,}5$ a $8/64 + 32/16\,384 = 0{,}125 + 0{,}00195 = 0{,}127$ bits por parámetro: un ahorro de 0,373 bits por parámetro. Antes de cuantizarlas, las constantes se centran restando su media, lo que permite una cuantización simétrica.

6.4 QLoRA

Definición 6.6 (QLoRA). QLoRA almacena los pesos congelados del modelo base en NF4 con doble cuantización y añade adaptadores LoRA en bf16 a las capas lineales. La pasada hacia delante de una capa es

$$Y = X\,\mathrm{dq}(c_1, c_2, W_{\mathrm{NF4}}) + s\,XL_1L_2,$$

donde dq descuantiza primero las constantes y después los pesos, produciendo una matriz en bf16, y L_1, L_2 son los factores LoRA [11, §3, ecs. 5 y 6]. El tipo de almacenamiento es NF4 y el tipo de cálculo, bf16. Solo se calculan gradientes respecto a los parámetros de LoRA.

En esta definición se usa el convenio de filas de [11], $Y = XW$; los factores L_1, L_2 desempeñan el papel de $A^\top$ y $B^\top$. Aunque W no recibe gradiente, la retropropagación sí necesita multiplicar por W descuantizada para propagar el gradiente hacia las capas anteriores, cuyos adaptadores también se entrenan.

El tercer ingrediente son los *optimizadores paginados*: el estado del optimizador se guarda en memoria unificada, que el controlador traslada automáticamente a la memoria de la CPU cuando la GPU se queda sin espacio y devuelve cuando hace falta para la actualización. Con estos tres elementos, Dettmers *et al.* ajustaron un modelo de 65 000 millones de parámetros en una sola GPU de 48 GB, frente a los más de 780 GB que requería el ajuste completo en 16 bits [11, §1]. Su mejor modelo, Guanaco 65B, alcanzó el 99,3 % del rendimiento de ChatGPT en la prueba de Vicuna según la evaluación de GPT-4, con un intervalo de confianza del 95 % de ±4,4 puntos. Los propios autores advierten que no demostraron que QLoRA iguale al ajuste completo en 16 bits a las escalas de 33 000 y 65 000 millones de parámetros [11, §8].

Proposición 6.7 (Memoria de pesos y optimizador). *Sin contar las activaciones, la memoria de pesos y estado del optimizador es de unos $16N$ bytes en el ajuste completo con Adam en precisión mixta y de unos $(b_{\mathrm{ef}}/8)\,N + 16N_{\mathrm{LoRA}}$ bytes en QLoRA, donde b_{ef} son los bits efectivos por parámetro del modelo base.*

La reducción es de unas $112/3{,}82 \approx 29$ veces y permite ajustar un modelo de este tamaño en una GPU de consumo. Con las constantes de escala, $b_{\mathrm{ef}} \approx 4{,}127$ en lugar de 4, lo que añade unos 0,11 GB.

Cuadro 6.1: Memoria de pesos y optimizador de un modelo de $7 \cdot 10^9$ parámetros, sin activaciones, tomando 4 bits por parámetro para la base cuantizada ($1\,\text{GB} = 10^9$ bytes).

Configuración	Cálculo	Memoria
Pesos en bf16	$7 \cdot 10^9 \times 2$ B	14 GB
Ajuste completo con Adam, precisión mixta	$7 \cdot 10^9 \times 16$ B	112 GB
QLoRA: base en 4 bits	$7 \cdot 10^9 \times 0{,}5$ B	3,5 GB
QLoRA: $2 \cdot 10^7$ parámetros LoRA	$2 \cdot 10^7 \times 16$ B	0,32 GB
QLoRA en total	$3{,}5 + 0{,}32$	3,82 GB

6.5 ¿Ajustar, dar instrucciones en el contexto o recuperar documentos?

El ajuste fino no es la única forma de adaptar un modelo. Las otras dos son dar instrucciones y ejemplos en el contexto (*prompting* y aprendizaje con pocos ejemplos) y recuperar documentos relevantes e insertarlos en el contexto (RAG, módulo 22). La regla práctica es que el ajuste cambia *cómo* responde el modelo y el contexto cambia *qué sabe* en cada consulta.

Cuadro 6.2: Criterios para elegir la forma de adaptación.

Criterio	Instrucciones en el contexto	RAG	Ajuste fino
Coste inicial	ninguno	índice de documentos	datos y entrenamiento
Conocimiento nuevo o cambiante	solo si cabe en el contexto	sí, con fuentes citables	mal: caro de actualizar y propenso a inventar
Formato, estilo y tono	aceptable	no ayuda	muy bueno
Coste por consulta	contextos largos	recuperación y contexto	contextos cortos
Riesgo de olvido	ninguno	ninguno	sí (tema 3)
Acceso a los pesos	no	no	sí, o una API de ajuste

En la práctica, las tres formas se combinan: un modelo ajustado para seguir un formato y citar sus fuentes, alimentado con documentos recuperados. Los proveedores de modelos comerciales ofrecen públicamente, según el caso, ajuste fino a través de sus plataformas; los detalles técnicos de esos servicios no siempre se publican.

Bibliografía

[1] A. Aghajanyan, L. Zettlemoyer y S. Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. En *Proceedings of ACL-IJCNLP*, 2021. https://arxiv.org/abs/2012.13255

[2] Y. Bai *et al.* Constitutional AI: Harmlessness from AI feedback. Anthropic, 2022. https://arxiv.org/abs/2212.08073

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. §4.4, «The Laplace Approximation».

[4] G. Neubig. 11-711: Advanced Natural Language Processing. Carnegie Mellon University, otoño de 2024. https://www.phontron.com/class/anlp-fall2024/

[5] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 8, «Post-training (RLHF, SFT, DPO)»; clase 9, «Efficient Adaptation (Prompting + PEFT)»; clase invitada de J. Schulman, «Tinker and LoRA Without Regret». https://web.stanford.edu/class/cs224n/

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2025. Clase 15, «Alignment – SFT/RLHF». https://cs336.stanford.edu/spring2025

[7] Stanford CS336. Assignment 1 (basics): Building a Transformer LM. Primavera de 2026. §3.5, problema `transformer_accounting`. https://github.com/stanford-cs336/assignment1-basics

[8] Stanford CS336. Assignment 5 (alignment): Alignment and Reasoning RL. Primavera de 2025. §4.2, problemas `tokenize_prompt_and_output`, `masked_normalize` y `sft_microbatch_train_step`. https://github.com/stanford-cs336/assignment5-alignment

[9] Stanford CS336. Assignment 5 Supplement: Instruction Tuning and RLHF. Primavera de 2026. §4.2, problema `data_loading`. https://github.com/stanford-cs336/assignment5-alignment

[10] M. P. Deisenroth, A. A. Faisal y C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. Versión en línea de 2024. Cap. 4 «Matrix Decompositions», §4.6 y ejercicio 4.10. https://mml-book.github.io/

[11] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.14314

[12] C. Eckart y G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[13] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[14] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. §7.1.1, «L^2 Parameter Regularization». https://www.deeplearningbook.org/

[15] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick y G. Neubig. Towards a unified view of parameter-efficient transfer learning. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2110.04366

[16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan y S. Gelly. Parameter-efficient transfer learning for NLP. En *International Conference on Machine Learning (ICML)*, 2019. https://arxiv.org/abs/1902.00751

[17] J. Howard y S. Ruder. Universal language model fine-tuning for text classification. En *Proceedings of ACL*, 2018. https://arxiv.org/abs/1801.06146

[18] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang y W. Chen. LoRA: Low-rank adaptation of large language models. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685

[19] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 8, «Post-training»; cap. 11, «Information Retrieval and RAG». https://web.stanford.edu/~jurafsky/slp3/

[20] J. Kirkpatrick *et al.* Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. https://arxiv.org/abs/1612.00796

[21] B. Lester, R. Al-Rfou y N. Constant. The power of scale for parameter-efficient prompt tuning. En *Proceedings of EMNLP*, 2021. https://arxiv.org/abs/2104.08691

[22] C. Li, H. Farkhoor, R. Liu y J. Yosinski. Measuring the intrinsic dimension of objective landscapes. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1804.08838

[23] X. L. Li y P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. En *Proceedings of ACL-IJCNLP*, 2021. https://arxiv.org/abs/2101.00190

[24] M. McCloskey y N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[25] L. Ouyang *et al.* Training language models to follow instructions with human feedback. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2203.02155

[26] V. Sanh *et al.* Multitask prompted training enables zero-shot task generalization. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2110.08207

[27] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai y Q. V. Le. Finetuned language models are zero-shot learners. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2109.01652

[28] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §14.2, «Fine-Tuning». https://d2l.ai/

[29] C. Zhou *et al.* LIMA: Less is more for alignment. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.11206